2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)

SegFold: Accelerating Sparse GEMM with a Fine-Grained Dynamic Dataflow

Xinrui Wu, Hanyu Wang, Jason Cong, and Tony Nowatzki
University of California, Los Angeles, USA
{alicewu,hanyuwang,cong,tjn}@cs.ucla.edu

***Abstract*—Generalized sparse matrix-matrix multiplication (SpGEMM) is critical in many domains. Existing CPUs and GPUs, as well as specialized accelerators, rely on static dataflows (e.g., inner product, outer product, Gustavson, etc.). Each static dataflow sacrifices some data reuse opportunities and imposes constraints on load balance.**

To address this inefficiency, we extend the typical SpGEMM dataflows by considering dynamism. Specifically, we add fine-grained dynamic scheduling to optimize reuse and reduce resource contention. We also develop dynamic remapping of partially completed work to improve load balance and parallelism. These ideas are formalized into a specific dataflow called Segment. To demonstrate Segment, we codesign a SpGEMM accelerator called SegFold. SegFold includes a memory controller that identifies fine-grained reuse opportunities in a local window of the stationary input array and exploits them through dynamic work assignment. It also incorporates a merge network that routes inputs to appropriate processing elements (PEs) for computation while dynamically remapping the work assigned to each PE to balance load.

Across diverse densities and matrix sizes, SegFold achieves a geometric-mean $1.95\times$ speedup over state-of-the-art SpGEMM accelerators and $5.3\times$ over the best static dataflow configuration, demonstrating that adding dynamism to the dataflow design space unlocks reuse and load-balance gains that no static scheduling choice can achieve in isolation.

***Index Terms*—SpGEMM, accelerator, dataflow, sparsity**

I. Introduction

Generalized sparse matrix-matrix multiplication (SpGEMM) is rapidly becoming the algorithmic backbone of workloads with dual-side sparsity, where both operands are sparse. Such dual-side sparsity arises not only in traditional domains—e.g., graph analytics, scientific simulation, optimization, and economic modeling—but also in modern machine learning, including pruned or structurally sparse DNN inference and sparse attention in large language models.

SpGEMM is a difficult workload to execute efficiently, as the computations are fundamentally data-dependent at a fine-grain, resulting in a host of challenges from irregular memory access, data-dependent control, load imbalance, and excessive bandwidth use. These challenges are particularly severe for unstructured sparsity, which is our focus. The vector execution models on CPUs and GPUs are not well-suited for these data-dependent computations, and thus a wide design space of specialized sparse matrix accelerators has been explored (e.g. [14], [16], [17], [23], [32], [33], [36], [38], [41]–[44], [53], [55], [56]). However, prior accelerators still have large headroom for improvement, as they either require high hardware overhead or do not achieve high utilization.

While the SpGEMM design space has been well formalized and explored (e.g. [7], [10], [11], [28], [31], [47]), the conventional taxonomy ignores a key dimension of dynamism, leading to lost opportunities in achieving high data reuse and good load balance across processing elements (PEs)—both of which are key to high efficiency.

Prior accelerators typically adopt one of three dataflows to process each tile: inner product, outer product, or Gustavson. Recent designs such as Spada [24], Trapezoid [49], and Flexagon [28] can select among these per tile, but the scheduling within each tile remains static. Additionally, each of the three dataflows makes different tradeoffs. Inner product reuses partial sums but sacrifices input locality and requires balanced row-column intersections. Outer product reuses inputs but sacrifices output reuse and requires balanced input nonzeros. Gustavson sacrifices reuse on one of the two inputs and imposes some load balance constraints on both nonzeros and intersections. The best approach depends on the sparsity pattern of the dataset [28], [45], [48], and even depends on sparsity within active tiles [24].

Our key insight is that higher reuse and better load balance can be achieved by introducing temporal and spatial dynamism into the dataflow for processing a single tile. First, instead of choosing a static ordering of rows and columns to process, a dataflow with *dynamic scheduling* can choose different fine-grained execution orders on different tiles to improve reuse and reduce resource contention. A dataflow with *dynamic mapping* can also re-partition storage dynamically to optimize for high PE utilization. These degrees of dynamism show that the current dataflow taxonomies are incomplete.

In this work, we propose a new dynamic dataflow and codesign an efficient accelerator. Our dataflow, called Segment, uses dynamic scheduling to select a set of elements and rows from input matrices to multiply in each cycle; this dynamic ordering enables Segment to achieve the benefits of both *outer product* and *Gustavson* dataflows. Segment further balances load through dynamic mapping of partial sums across PEs; PEs redistribute partial sums across the array based on intersection patterns while preserving column ordering. Both aspects enable high utilization.

To implement this novel dataflow, we develop our microarchitecture named SegFold. SegFold comprises three key strategies: the memory controller (for dynamic scheduling),

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00130

the adaptive merge network (for dynamic mapping), and the folding mechanism (for virtualizing hardware resources). For dynamic scheduling, the memory controller maintains an active window to monitor multiple input rows and assign resources to maximize reuse within the window on-the-fly. The merge network enables dynamic mapping by redistributing each computation to an available PE based on the current distribution of partial sums. Finally, output rows vary in length: long rows can overflow a single physical PE row, while short rows leave neighboring PE rows underutilized. We implement a folding mechanism that maps overflow portions of long output rows onto neighboring physical PE rows whose resident output rows are shorter, improving utilization without requiring each logical row to fit within one PE row.

We created a detailed cycle-level simulator for SegFold, and implemented its major components in RTL for resource evaluation. We evaluate SegFold on a diverse set of matrices with different sizes and sparsity levels. We compare SegFold against state-of-the-art SpGEMM accelerators that support multiple static dataflows [28] or adaptive static dataflows [24]. Overall, SegFold achieves a geometric-mean speedup of $1.95\times$ over Spada and $5.3\times$ over the best Flexagon dataflow configuration.

To summarize, our main contributions are:

- A novel dataflow aspect: dynamic scheduling that reorders work in an active window to maximize data reuse.
- A novel dataflow aspect: dynamic mapping that distributes partial sums across PEs to balance load while preserving column ordering.
- A codesigned efficient microarchitecture with significant speedups over conventional static dataflows across a range of input matrices and sparsity levels.

II. Background

A. SpGEMM Dataflow Design Space

We use A, B to denote the input of SpGEMM, and C for the output. The arithmetic representation for SpGEMM is:

$$C_{m,n} = \sum_k A_{m,k} \times B_{k,n}, \tag{1}$$

where both A and B are sparse. The iteration space

$$S = \{(m,n,k) \mid 0 \le m < M,\ 0 \le n < N,\ 0 \le k < K\},$$

can be broken into multiplications and additions as:

$$T_{m,n,k} = A_{m,k} \times B_{k,n}, \tag{2}$$

$$C_{m,n} = \sum_k T_{m,n,k}. \tag{3}$$

T is the product of A and B elements paired by the shared index k; C is the reduction of T along the K dimension.

The major aspects of SpGEMM's dataflow design are:

1) Tiling strategy for M, N and K.
2) Scheduling of all dimensions (M, N, K plus tiling subdimensions) onto spatial and temporal axes.
3) Fusion of the computation in Eq. 2 and Eq. 3.

Our work focuses on the second and third aspects – i.e., the scheduling and fusion of work at a fine-grain within a tile. In this paper, we discuss three representative dataflows—inner product, outer product, and Gustavson. We next explain how each dataflow uses different scheduling and fusion strategies.

TABLE I: Taxonomy of SpGEMM accelerators.

Accelerator	Scheduling	Reconfigurability to Balance Load & Comp.
TPU [21]	–	–
ExTensor [17]	IP	–
SIGMA [39]	IP	Reconfigurable reduction network
OuterSpace [33]	OP	None
SpArch [54]	OP	Data-dependent comparator arrays
MatRaptor [43]	Gust	Comparator queues
Gamma [51]	Gust	None
Flexagon [28]	IP/OP/Gust	Reconfigurable distribution, merge, and reduction networks
Trapezoid [49]	IP/Gust (spatial/temporal)	Reconfigurable distribution, merge, and reduction networks
Spada [24]	Window/Tile-size Adaptive	Neighboring-lane work stealing
SegFold (ours)	Sub-tile Dynamic dataflow	Sub-tile scheduling & Reconfigurable merge network

Static Dataflow Definitions. *Inner product* (order $M \to N \to K$) performs dot products between rows of A and columns of B. *Outer product* (order $K \to M \to N$) performs cross products between column vectors from A and row vectors from B. *Gustavson* (order $M \to K \to N$) performs row products between elements $A_{m,k}$ and the corresponding k row from B. Figure 1 shows an example of these dataflows, which we discuss further in §II-C to elucidate their limitations.

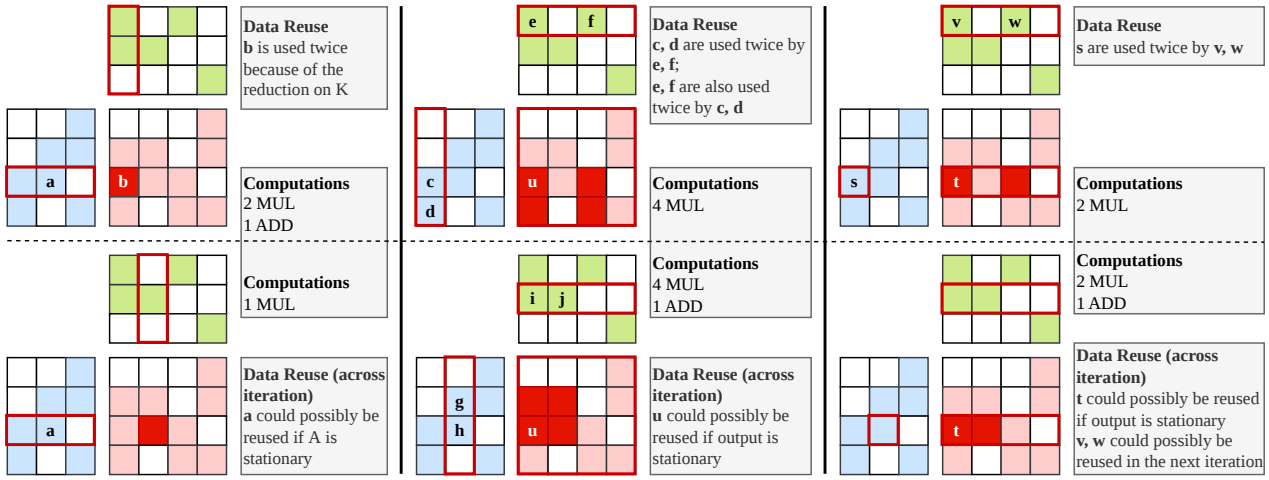
B. Dataflow in Prior Work

SpGEMM accelerators span a diverse design space, characterized by their dataflow strategies and degrees of hardware reconfigurability (see Table I). Early versions of TPU [9], [21] use weight-stationary systolic arrays optimized for dense workloads, without sparsity support.

In the sparse domain, most prior accelerators adopt a static dataflow. Architectures like ExTensor adopt a fixed inner-product dataflow [17]. SIGMA builds upon this by incorporating a reconfigurable reduction network [39]. Outer-product accelerators, such as OuterSpace [33] and SpArch [54], exploit sparsity using mechanisms like specialized memory hierarchy or comparator arrays. Gustavson-inspired designs, including MatRaptor [43] and Gamma [51], focus on row-/column-wise intersections through comparator queues; Gamma and Zed [6] additionally include a matrix-preprocessing step that groups similar rows of the stationary matrix to improve reuse on the streaming matrix.

Recent efforts like Flexagon [28], Trapezoid [49], SpMARD [46], SparGD [45], and SPARM [26] support multiple dataflows (inner-product, outer-product, Gustavson), employing reconfigurable distribution, merge, and reduction fabrics for greater flexibility across different matrices.

Spada [24] develops a window-adaptive (WA) dataflow that supports a spectrum of execution modes by adjusting window height and width at tile granularity to realize the input and output reuse benefits of different dataflows under varying sparse patterns. It uses local dynamism: neighboring lanes can opportunistically process each other's elements when idle, providing some degree of dynamic load balancing within



(a) Example of Inner Product (b) Example of Outer Product (c) Example of Gustavson

Fig. 1: Comparison of classic dataflows. Every two consecutive blocks in a column (top vs bottom) are two sequential snapshots.

the merge network. However, the overall scheduling remains statically determined by the tiled loop structure.

Within this continuum, SegFold introduces a dynamic dataflow that adapts at sub-tile granularity. Unlike Spada's window-size adaptation, SegFold dynamically reorders work selection (SELECTA in §III-A) within an active window based on instantaneous reuse opportunities, and uses a reconfigurable merge network to dynamically remap partial sums (SEGMENTBC in §III-B) based on output sparsity patterns discovered on-the-fly.

C. Limitations of Static Scheduling: Low Data Reuse and Imbalanced Load & Computation

Data Reuse. The scheduling order of M, N, and K determines the reuse of A, B, and C. Input reuse and output reuse are both governed by where K sits in the loop nest. The further outward K is placed (relative to M or N), the more reuse of B or A, respectively. Conversely, the further inward K is placed, the more reuse of C.

For *inner product*, K is in the innermost loop; therefore, only outputs are reused, as shown in a single iteration in Fig. 1(a). However, there is possible data reuse of A across iterations if we consider A to be stationary. For *outer product*, K is in the outermost loop, so both A and B exhibit good data reuse. However, each iteration generates an entire $T_{M,N,k}$, meaning that revisiting the same output entry is distant in the loop nest. The upper bound (dense) distance between two outputs that must be accumulated is $M \times N$, and this distance varies based on the occupancy of A and B.

For *Gustavson*, K is in the middle loop and N is in the innermost loop. This allows A to be fully reused. The output reuse is less distant than in *outer product*, since each iteration generates only a row of the partial sum. The upper bound (dense) distance between two accumulated outputs is N. However, this dataflow sacrifices the potential for good reuse of B and still faces the variable intermediate output sizes. Therefore, no single static schedule can simultaneously maximize reuse of all inputs and outputs.

Imbalanced Load & Computation. Static scheduling exacerbates imbalance in load and computation, especially when there is non-uniformity in the data. For *inner product*, static scheduling always computes between a row vector and a column vector. The actual number of intersections is determined by the nonzero positions in the two vectors. As shown in Fig. 1(a), the number of multiplications and additions varies across iterations. For *outer product*, static scheduling causes each iteration to generate an entire matrix as a partial sum. The number of multiplications depends on the nonzeros in the column and row. The number of additions depends entirely on the positions of sparse values in A and B. As shown in Fig. 1(b), there is only one addition across the matrix, while other positions have zero additions. For *Gustavson*, the computation imbalance is similar to that of *outer product*. Additionally, static scheduling causes load imbalance across A rows, which further harms the reuse of B rows, as shown in Fig. 1(c).

D. Motivation for Dynamic Dataflow

Static scheduling fixes the execution order and resource assignment ahead of time, leaving it unable to react to variations in nonzero distribution or rebalance work at runtime. As a result, no static dataflow can simultaneously maximize reuse on all three operands. We propose a dataflow that integrates *dynamic scheduling* to adapt work selection within a tile and *dynamic mapping* to redistribute partial sums across PEs at runtime, broadening the achievable reuse–utilization tradeoff.

III. SEGMENT DATAFLOW

Compared to prior dataflows that follow a fixed loop order and iterate strictly over static loop bounds, we propose a new dynamic dataflow, which we call *Segment*. Segment dataflow uses an active window to reorganize iteration over $A_{m,k}$ to

maximize reuse and load balance. To enable efficient management of irregular intersections, we maintain partial results in a compressed, ordered coordinate space mapped to PEs. We use the term *segment* because it represents the bounded path that an element takes through this coordinate space from injection to accumulation.

Figure 2(d) summarizes the algorithm. It introduces two key components that add dynamism:

1) SELECTA: dynamically selects and reorders (m, k) pairs, breaking static m–k nesting to increase B reuse and spatial parallelism, while allowing partial B-row processing to absorb row-wise irregularity. Each invocation produces up to $R_{\max}$ (the PE-row capacity) pairs, where multiple pairs may share the same k to enable B-row reuse but no two share the same m to avoid C-row contention.
2) SEGMENTBC: performs on-the-fly B–C intersections across segments, exploiting spatial/temporal locality in C to improve reuse of C and reduce irregular reduction overhead. It consumes the selected pairs and their partial B rows from SELECTA and routes each $B_{k,n}$ element to find or create its position in a coordinate space of C.

The main advantage of Segment dataflow is its ability to simultaneously exploit reuse across all three operands while increasing parallelism by combining multiple granularities within a unified execution model. Specifically, it captures element-wise reuse of A, row-wise reuse of B across different A elements, and tensor-wise reuse of C across different partial-sum iterations. In addition, Segment dataflow provides element-wise flexibility in C reductions by allowing elements within the tensor to move dynamically, thereby redistributing the reductions to allow parallelization. Canonical sparse dataflows do not provide this combination of properties. IP only exploits element-wise reuse of C; OP enables row-wise reuse of A and B but suffers from poor C reuse and high irregular reductions; Gustavson achieves element-wise reuse of A and row-wise reuse of C, but misses opportunities for reusing B and still incurs significant reduction irregularities on C.

SegFold realizes this dataflow through SELECTA and SEGMENTBC, as illustrated in Fig. 2(d). Matrix A is processed at element-wise granularity, while matrix B is processed at row-wise granularity. In each iteration, SELECTA examines the active window over A and greedily selects (m, k) pairs that maximize row-wise reuse of B. The selected A elements are then broadcast together with their corresponding B rows to exploit element-wise reuse. For each $B_{k,n}$ element, SEGMENTBC dynamically routes it within a row of the tensor-wise C, either accumulating it into an existing partial sum or inserting a new entry. Across consecutive iterations, reductions for C elements are redistributed dynamically to mitigate register pressure and improve load balance.

A. SELECTA: *row-wise intersection with reordering* K

Because matrix B is processed at row-wise granularity, dimension N is placed in the innermost loop, as shown in Fig. 2(d). This inherently maximizes the reuse of A. The nonzero positions of A determine which rows of B are accessed in each iteration. SELECTA therefore picks a set of A elements that maximizes **row-wise intersection**—the number of m indices sharing the same k—which directly increases reuse of the corresponding B row.

Algorithm 1: SELECTA: Dynamic (m, k) Selection

Input: A bitmask, B-row metadata, hardware state; $W_{\max}$ window size; $R_{\max}$ PE-row capacity
Output: Selected (m, k) pairs, corresponding partial B rows

```
// Inter-tile: sliding window over K
1  while |W_k| < W_max and HASMOREK() do
2  |  W_k ← W_k ∪ {next k};
3  end
   // Intra-tile: greedy mk-dynamic
      selection
4  selected ← ∅;  usedM ← ∅;
5  foreach k ∈ W_k do
6  |  if |selected| ≥ R_max then break;
7  |  foreach parallel m s.t. A[m,k] = 1 do
8  |  |  if m ∉ usedM and |selected| < R_max then
9  |  |  |  selected ← selected ∪ {(m,k)};
10 |  |  |  usedM ← usedM ∪ {m};
11 |  |  end
12 |  end
13 end
   // Inter-tile: Retire completed ks and
      refill window
14 foreach k ∈ W_k s.t. ALLDONE(k) do
15 |  W_k ← W_k \ {k};
16 end
17 return selected;
```

Algorithm 1 formalizes SELECTA: it selects (m, k) pairs from the input metadata based on the runtime hardware state. The flexibility in SELECTA originates from the associativity of the reduction over the K dimension, which allows intersections in Eq. 2 to be computed in any order without violating the accumulation rule in Eq. 3. Based on this observation, SELECTA performs two levels of reordering: intra-tile reordering, which maximizes the number of A entries selected within each k column, and inter-tile reordering, which maximizes the number of k columns considered concurrently.

Intra-tile Reordering. Unlike prior dataflows that follow either an m-prior or a k-prior order, SELECTA decides the set of (m, k) pairs using an mk-dynamic order. More specifically, we simultaneously consider the following two criteria. First, we greedily maximize the number of pairs that share the same k, as shown in Algorithm 1 line 5. Second, we avoid selecting pairs with different k values but the same m index, as shown in Algorithm 1 line 8, because they update the same output row and can create reduction conflicts when their generated partials share column positions.

We use the A pattern in Fig. 2 to demonstrate the advantages of our algorithm over conventional ones. Gustavson dataflow (Fig. 2(a)) follows an m-prior order and chooses $A_{0,2}$, $A_{1,1}$, $A_{2,0}$ and $A_{3,0}$. This restricts the potential reuse of the first B row, corresponding to the two elements highlighted in the yellow box. Outer product (Fig. 2(b)) follows a k-prior order

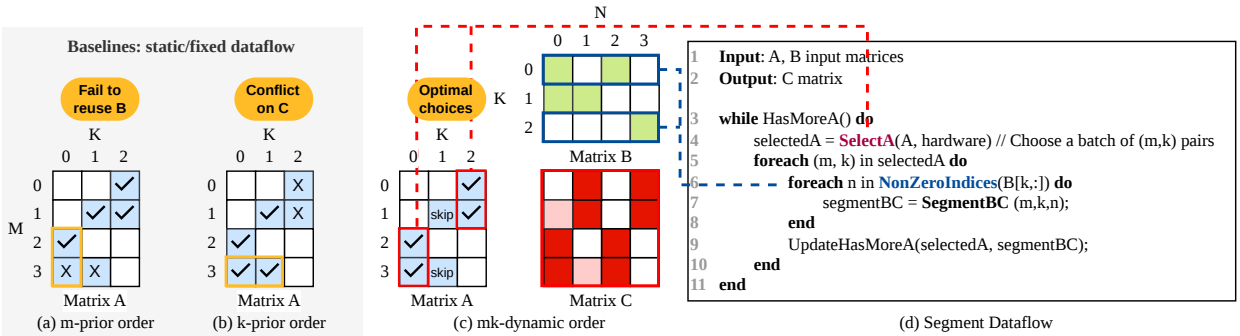


Fig. 2: SegFold Dataflow Example. A batch of (m, k) pairs is chosen by SELECTA, and SEGMENTBC(m, k, n) is invoked over $B[k, :]$ nonzeros for ordered processing in the virtual coordinate space. Reuse benefits over two iterations are shown in pink. Tick marks indicate the A elements selected for each spatial iteration under different dataflows.

and selects $A_{2,0}$, $A_{3,0}$, $A_{1,1}$ and $A_{3,1}$. Nevertheless, $A_{3,0}$ and $A_{3,1}$ have the same m index, so their products target the same output row and may contend when the corresponding B rows contribute to overlapping n, which we do not know beforehand. Meanwhile, our algorithm selects $A_{0,2}$, $A_{1,2}$, $A_{2,0}$, and $A_{3,0}$, exploiting B reuse while avoiding reduction conflicts in C.

Inter-tile Reordering. Instead of choosing (m,k) pairs from a fixed tile, SELECTA maintains a *sliding window* over the K dimension. A k value in the window is marked as complete and replaced by a new one once all A–B intersections for that k have been processed.

For example, in Fig. 3, the third column of A and the corresponding third k slice of B (shown transposed in the figure) each participate in only one effective intersection. Once that intersection completes, the window advances from $k = \{0, 1, 2\}$ to $k = \{0, 1, 3\}$. This enables k-level pipelining across different (m, k) tiles.

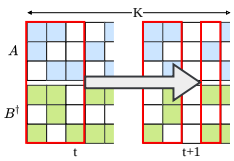


Fig. 3: Example active window of SELECTA, shown in red for two time steps.

Moreover, using an entire B row as the intersection granularity increases reuse of the triggering A element across the loaded B row, but it also reduces the available parallelism across different B rows, because it requires the dataflow to finish an entire row of length N before moving on to the next intersection. To alleviate this limitation, we relax the requirement of fully loading a B row before processing and instead allow the hardware to operate on *partial* B rows. Concretely, we decompose the N dimension into smaller segments and interleave segments from multiple B rows. A single active window tracks the progress of each B row within the window, enabling higher inter-row parallelism while preserving the reuse benefits of row-wise intersection.

B. SEGMENTBC*: on-the-fly element-wise redistribution*

The sparse patterns of both T and C are highly irregular and data-dependent, making it impractical to materialize a large intermediate tensor T and then reduce it afterward. Therefore, Segment dataflow performs on-the-fly element-wise redistribution to (1) generate and maintain compressed indices for C and (2) reduce each $T_{m,n,k}$ in place into the partial sum $C^*_{m,n}$[1].

Let $(x, y) \in X \times Y$ denote the virtual coordinate at which a nonzero element of C is allocated for reduction, where $|X|$ is the number of non-empty rows in C and $|Y|$ is the maximum number of nonzeros in any row of C. We refer to $\mathcal{V} = X \times Y$ as the *virtual coordinate space*; each occupied virtual coordinate holds a distinct C element. As new $T_{m,n,k}$ are deposited into $C^*_{m,n}$, the virtual coordinate space evolves over time: newly created C entries are inserted, shifting existing entries to maintain ordering. In the $\mathcal{V}$ space and the algorithm description, we do not distinguish T from B, since the dynamics depend only on the n indices. We use f_t to denote the mapping from the dense Cartesian space to the $\mathcal{V}$ space at time step t:

$$f_t(m, n) = (x, y). \tag{4}$$

$f_t(m, n)$ indicates the compressed virtual location that stores the partial sum $C^*_{m,n}$ at time t.

To preserve both correctness and maximize locality, we constrain the mapping f_t with the following properties:

1) **Injectivity.** f_t is injective: partial sums for distinct (m, n) pairs are assigned to distinct virtual coordinates (x, y).
2) **Row saturation.** Within each virtual row, nonzeros occupy consecutive y positions, left to right (no gaps).
3) **Column ordering.** Within each virtual row, column indices strictly increase from left to right. That is, $\forall m$, if $f_t(m, n_1) = (x, y_1)$ and $f_t(m, n_2) = (x, y_2)$ with $n_1 < n_2$, then $y_1 < y_2$.
4) **Time ascending.** Entries in a virtual row only move "forward" over time. For each x, if $f_t(m, n) = (x, y)$ and $f_{t'}(m, n) = (x, y')$ with $t < t'$, then $y \leq y'$.

Fig. 4 illustrates how the mapping f is updated in the walk-through example from Fig. 2(c). The numbers in the boxes show the column index n in the dense Cartesian space. At time step t, the bottom row of f_t contains $\{0, 2\}$ (Fig. 4(a)). When new B elements arrive and intersect with matching A elements, they generate partial sums with column indices

[1]Here we use * to denote the intermediate result accumulated so far.

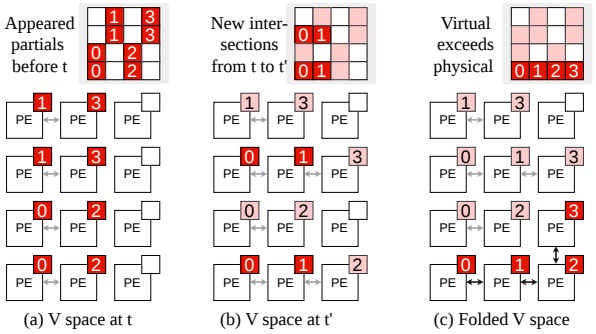


Fig. 4: Example of on-the-fly update of the mapping from the C column indices to the $\mathcal{V}$ space. Example from Figure 2(c).

$\{0, 1\}$. Because the partial sum with column index 1 has not previously appeared in the $\mathcal{V}$ space, the mapping is updated to $\{0, 1, 2\}$ to satisfy the column-ordering property (Fig. 4(b)).

With the definition of the $\mathcal{V}$ space, we formally define *segment* as the displacement, measured in virtual coordinate positions, that a B element traverses before reaching its final position in the PE array. The name SEGMENTBC reflects that an element enters the $\mathcal{V}$ space as B carrying B's metadata and leaves as C with its value and metadata updated. Specifically, for a B element with column index n that enters the array and is eventually accumulated into row m of C:

$$\text{displacement} = \|f_{t_{\text{in}}}(m, n) - f_{t_{\text{out}}}(m, n)\|, \tag{5}$$

where t_{in} and t_{out} denote the times when the element enters and is consumed, respectively. The displacement arises because the $\mathcal{V}$ space mapping f evolves over time: as new A–B intersections are discovered on-the-fly, newly formed C^* entries are inserted into the virtual coordinate space, shifting existing entries and potentially increasing the distance from $f_{t_{\text{in}}}$ to the final position $f_{t_{\text{out}}}$. Additionally, the initial mapping $f_{t_{\text{in}}}$ may be approximate due to hardware constraints (§IV-A2).

The optimization goal of SEGMENTBC is to minimize segment displacement, since longer displacements cause more network contention. We employ a dynamic mapping strategy that determines where B elements are placed and how they traverse the array; we discuss data traversal in §IV-A1 and mapping in §IV-A2.

IV. SEGFOLD MICROARCHITECTURE

Figure 5 presents a high-level overview of the SegFold microarchitecture. SegFold adopts a two-dimensional array of processing elements (PEs), following the architectural principles of prior spatial accelerators [3], [21], [37], [49]. In terms of the stationarity taxonomy [31], SegFold is effectively *row-stationary* at the PE-row granularity, where each row owns an entire virtual row of C for its lifetime, but only *approximately* output-stationary at the PE level, since outputs may be remapped between PEs by spatial folding or spad spills. This two-level view captures SegFold's tradeoff: strict stationarity at the finest grain is sacrificed for the flexibility to rebalance load at runtime.

To efficiently orchestrate data movement, SegFold integrates a hybrid on-chip interconnect. At the single-PE level, each PE communicates with its nearest neighbors, forming a mesh network across the 2D PE array. At the PE-row level, each PE row has a dedicated *row shifter* that shifts and streams the partial B row horizontally into the PEs. Across PE rows, a *vector multicast network* routes B segments to the appropriate PE rows, enabling flexible and conflict-free global redistribution. Together, these components provide both fine-grained local communication and high-bandwidth global redistribution essential for SegFold's dynamic dataflow.

To enable high intra-tile reuse, SegFold employs an on-chip cache to store tiles of A and B. §IV-B describes the memory controller, which is the key component realizing the Segment dataflow. As shown in Figure 5, this unit is connected to the on-chip metadata buffer and to the Index-to-PE Mapper (IPM)—a lookup table that maps each incoming B element's column index to its starting PE position in the merge network (§IV-A2), enabling it to perform the SELECTA scheduling algorithm and SEGMENTBC mapping algorithm respectively. It ensures that the PE array will receive a steady, reuse-optimized supply of matrix tiles and minimizes off-chip memory accesses.

A. PE Row

Figure 5(b) illustrates a representative PE row containing four PEs. Each PE integrates an arithmetic logic unit (ALU) for local computation, a FIFO buffer that stores matched-but-not-yet-consumed B values, and a router with four physical ports to its neighbors. Although the router supports four-way connectivity, only one direction is active at any time, determined by the folding mechanism (§IV-D).

Initially, the router's active direction is *rightwards*, consistent with the default column-ordering rule of the $\mathcal{V}$ space mapping. Here, the PE row behaves as a simple right-propagating merge network, forming the baseline for our design. The folding technique (§IV-D) modifies these active directions.

We first introduce our PE-to-Virtual-Coordinate mapping. Each PE in a row holds exactly one virtual coordinate position of the output matrix C. Concretely, PE p in row r stores the partial sum $C^*_{m,n}$ whose $\mathcal{V}$ space coordinate is (r, p), along with the Cartesian column index n that maps to this position. A PE row therefore represents one virtual row of C, with PEs ordered left-to-right by increasing column index. When a B element with column index b enters a PE row, it is injected at the starting position determined by the IPM (§IV-A2) and traverses rightward through the merge network, comparing b against each PE's stored column index c until it finds a match ($b = c$, triggering accumulation) or a gap ($b < c$, triggering insertion).

Each PE row also has a dedicated connection to a small local memory containing: (i) the IPM, which the merge network keeps up-to-date as C entries shift, and (ii) a scratchpad (spad) holding overflow C values. All PEs within a row share this memory interface; therefore, we limit accesses to avoid contention.

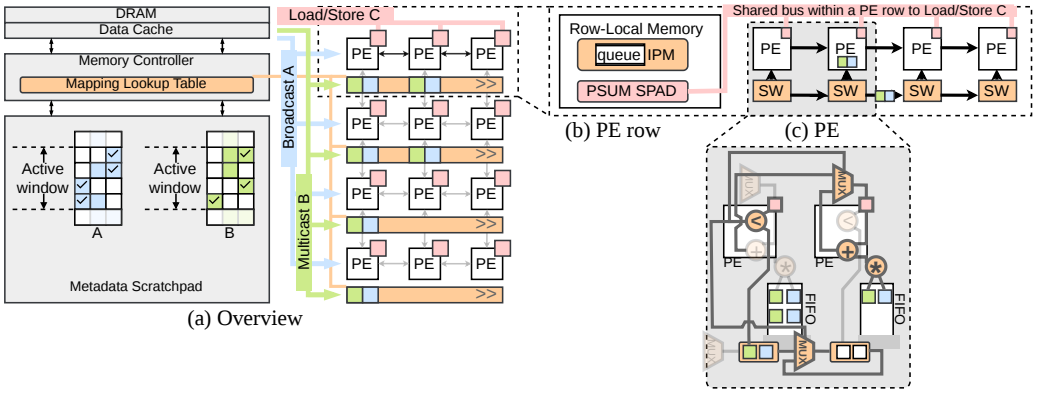


Fig. 5: SegFold μarch overview. PEs communicate over a local network, and share a row-level memory.

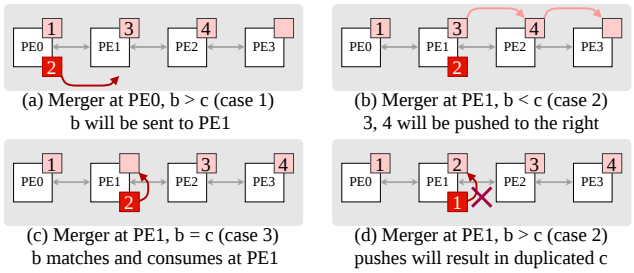


Fig. 6: Example of on-the-fly intersection where the indices of B are $>$, $<$, $=$, and conflicting with the C index.

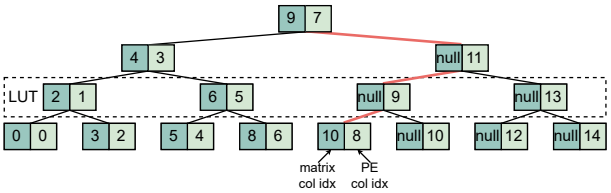


Fig. 7: Index to PE mapper (IPM) using binary search. Each level is in its own look up table (LUT) for pipelining.

1) Adaptive Merge Network: We refer to the PEs' local interconnect as a *merge network*. Each PE contains a *merger* (the local component of the merge network) that compares the incoming B element's column index against the C-column index stored at this $\mathcal{V}$ space position. Based on this comparison, the merger decides whether to forward b to a neighbor or retain it locally for accumulation.

Figure 6 illustrates how the merge network determines the final virtual position $f_{t_{\text{out}}}$ in SEGMENTBC for each incoming B element. We use the red boxes to denote the Cartesian column index of the incoming B element and the pink boxes to denote the Cartesian column index of the C^* element stored at the PE.

Consider a PE row where the C^*-column indices $\{1, 3, 4\}$ are stored at positions $y = \{0, 1, 2\}$. A B element with column index $b = 2$ arrives at $f_{t_{\text{in}}} = 0$. At each position, the merger compares b and c (we use lower case to represent column indices), leading to one of these cases:

- **Fig. 6(a):** $b > c$**.** Because the C^*-column indices are strictly monotonic, the final position $f_{t_{\text{out}}}$ must lie to the right of y. The element is therefore forwarded to $y+1$.
- **Fig. 6(b):** $b < c$**.** If we eventually reach a position y with a larger c, because monotonicity ensures that no match exists further to the right, the network shifts all C^*-column indices to the right by one slot. This creates an empty slot at y: the final virtual position $f_{t_{\text{out}}}$.
- **Fig. 6(c):** $b = c$**.** A match is found at position y, which becomes $f_{t_{\text{out}}}$.

Together, these cases ensure that each B element successfully finds its final virtual position $f_{t_{\text{out}}}$, provided that the C^*-column indices to the left of $f_{t_{\text{in}}}$ satisfy $c < b$. Figure 6(d) shows a scenario that violates this, and is prohibited by our dataflow.

2) Index to PE Mapper (IPM): To avoid the prohibited scenario in Fig. 6(d), the key challenge is to construct a mapping that (i) guarantees a legal $f_{t_{\text{in}}}$ for every segment and (ii) simultaneously minimizes the traversal distance required. Achieving both goals increases PE utilization by enabling more pipelined segment injections from different B rows, while also reducing merge-network latency and contention.

We will now use s as shorthand for $f_{t_{\text{in}}}$ (where the B element enters the network). As discussed, a starting point s is legal if all C^*-column indices that appear to the left of s are strictly smaller than b, i.e., $\forall i < s,\ c_i < b$. Due to row saturation and the column-ordering property, the c_i values stored in the merges are guaranteed to be strictly ordered from left to right without gaps. This monotonicity allows us to perform a binary search over the c_i values to determine the rightmost legal starting point for b.

Figure 7 shows an example of an IPM with multiple lookup tables (LUTs), looking up a B element with column index 11. In the first level, it is greater than 9 and will go to the right branch. In the second and third level, as both keys are showing null, it will go to the left branch. Finally, it reaches a leaf with value 8. Therefore, this b element will be mapped to the 8th merger and traverse the adaptive merge network.

Row-wise Mapping. Computing a separate mapping for every b element in a B row would be prohibitively expensive in

hardware, since it would require the IPM to support a number of simultaneous reads proportional to the number of PEs. To reduce this overhead, we instead compute the mapping only for the *first* nonzero element of each B row. This optimization is valid because, under the row mapping, each element's start point s increments by 1, matching the increment of its column index b. Combined with the column-ordering property, this guarantees that if the first mapped element of a row satisfies the legality condition $\forall i < s,\ c_i < b$, the remaining elements automatically satisfy it as well.

IPM Updates. When a PE updates its c value, it notifies the IPM. Because the LUT has a limited number of write ports, multiple updates to the same entry are queued and applied serially, so the IPM may not always reflect the most recent c value. Due to the time-ascending property of these updates, an out-of-date LUT can only map a b element to a position to the *left* of its true most-recent legal starting point. This does not violate correctness, since the merge network still identifies the correct match, but it may increase the segment length. We quantify the gap between fully up-to-date c indices and the LUT-based mapping with bounded write bandwidth in §VI-C.

B. Memory Controller

The memory controller in SegFold realizes the scheduling logic of the Segment dataflow: it maintains an active window over B rows, filters rows by intersection, and tracks partial-row progress, while also serving the underlying load/store requests for A, B, and C. Since the goal of SELECTA is to choose multiple A values from the same column, A is stored in a column-major format. In contrast, B is processed at row granularity, so it is stored in a row-major format. Only nonzero elements of both matrices are stored. Because accesses typically touch consecutive elements within a column of A or a row of B, the memory controller includes a coalescing unit that merges fine-grain requests before issuing them to the cache and DRAM.

As discussed in §III-A, the active window tracks a bounded set of K values for both A and B. The metadata for A is a compact bitmask recording consumption status. The SELECTA logic scans this A-side bitmask over the active window each cycle to avoid m-index conflicts among already-selected pairs and to maximize the nonzero k-index within a single column. The bitmask read and window scan are $O(W)$ in the window size; for our default $W = 32$, the entire scan completes within a single cycle as a modest combinational circuit.

For B, we use a doubly compressed sparse-row format (DCSR) [1], augmented with an extra start pointer per active row to track unprocessed elements. The second level of compression skips empty rows in $O(1)$ during scheduling, which is critical for highly sparse matrices where many rows in the active window contribute no nonzero intersections with the selected A columns and would otherwise be enumerated explicitly.

Supporting intra-window reordering of k means that a larger window enables more B rows to be processed in parallel. However, a larger window increases metadata size and update traffic. In §VI-D, we evaluate different window sizes and select a configuration that preserves performance while capping metadata overhead.

C. Vector Multicast Network

SegFold employs a vector multicast network [50] for loading B rows. Each PE row is responsible for processing at most one B row at a time. Because elements within a B row must be mapped to a consecutive set of mergers—with the head position determined dynamically—we incorporate a row shifter that realigns the incoming B elements to the appropriate consecutive positions. Furthermore, SELECTA may choose multiple distinct k values simultaneously, enabling several B rows to be active in the same cycle. As shown in Fig. 5(a), the network for B must multicast multiple B rows into the PE rows; we therefore use a vectorized crossbar to provide the multicast bandwidth from the memory backend to the PE rows. Because streaming multiple vectors increases complexity at the network ports, a key design parameter is the number of B rows that can be multicast in parallel. Our design allows up to 4 vector multicasts, which balances area/energy efficiency and performance.

D. Folding: Mapping Segments to PEs in Space and Time

The final $\mathcal{V}$ space representation of C will exhibit highly irregular row lengths, and each virtual position may require a different number of MAC operations. To maximize utilization and bandwidth, we do not constrain the $\mathcal{V}$ space row length to be less than the physical length of a PE row. Instead, we introduce two complementary techniques—*spatial folding* and *temporal folding*—that effectively map an irregular $\mathcal{V}$ space onto a regular 2D PE array.

Spatial folding addresses the mismatch between the irregular, variable-length rows in $\mathcal{V}$ space and the fixed physical width of the PE array. A long virtual row of C may extend beyond the capacity of a single PE row, while a short row may leave many PEs idle. To avoid underutilization, SegFold allows each logical row of outputs to *fold* across multiple PE rows.

Consider a PE array with R rows and P columns, indexed by coordinates (r, p) for $0 \le r < R,\ 0 \le p < P$. Each router maintains a direction configuration, which specifies which of the four output ports will be used to place the next logical column of the $\mathcal{V}$ space row. When an on-the-fly intersection generates a new virtual column j in an occupied physical position (r, p), the router at (r, p) examines its four neighboring PEs in the following priority: {right, up, down, and left}. Right is always the first priority to first realize the default merge network configurations.

Formally, it selects the first coordinate

$$(r', p') \in \big\{(r, p+1),\ (r-1, p),\ (r+1, p),\ (r, p-1)\big\} \quad (6)$$

whose occupancy bit satisfies $\mathcal{O}_{r',p'} = 0$. If such a neighbor exists, the router sets $\mathcal{O}_{r',p'} \leftarrow 1$, forwards the virtual column j to that PE, and updates its direction configuration to point to (r', p') for the next placement. In the example of Fig. 4(c), the fourth element with index 3 in the bottom row is folded to the

upper PE with an updated router configuration. Because each router exposes only its highest-priority free neighbor as the active direction, the spatial footprint of the $\mathcal{V}$ space row grows smoothly: horizontally when possible, and folding vertically when the row exceeds the local PE-row capacity. Through this mechanism, spatial folding enables irregular, long $\mathcal{V}$ space rows to occupy additional PE rows when necessary, while allowing shorter rows to leave unused PEs available for other work, thereby improving overall array utilization.

Temporal folding handles overflow and imbalance in the C reductions. While spatial folding effectively manages irregular row lengths and maps them onto the regular PE mesh, it does not guarantee that the number of nonzeros in a virtual row is always bounded by the total PE-array capacity. To support rows that exceed this capacity, each PE row is equipped with a spad that stores overflow virtual rows.

When a PE needs to accommodate a new partial sum, it can overwrite its local c_i register in place and spill the previous partial sum into the spad. This mechanism not only supports rows that are longer than the physical array, but also helps mitigate reduction imbalance: partial sums that are likely to complete early can be offloaded to the spad, allowing PEs to be reused for other work while preserving correctness.

E. Scalability

SegFold's control complexity scales linearly with array size in the dominant terms. The merge network traversal within each PE row is $O(P)$ in the worst case, where P is the number of PEs per row; however, the expected traversal is much shorter because the IPM provides a near-optimal starting position via $O(\log P)$ binary search, whose latency could be amortized through pipelining. The IPM itself uses a tree-structured lookup table whose depth grows as $\log P$, and whose total storage scales as $O(P)$ entries per row. The spad for overflow C values scales linearly with the number of PE rows (R), as each row maintains an independent spad. Overall, scaling the array from P to $2P$ PEs per row doubles the merge network width and IPM size, but does not change the asymptotic complexity of the control logic.

V. Methodology

Simulation Infrastructure. We evaluate SegFold using a cycle-level microarchitectural simulator. Hardware parameters are summarized in Table II. To maintain a hardware cost comparable to state-of-the-art sparse accelerators [28], SegFold instantiates a 16×16 2D PE array. The design meets timing at 1 GHz. The memory controller uses a fixed active-window size of 32, chosen to balance performance against metadata and storage overhead.

The memory hierarchy uses an on-chip cache backed by off-chip HBM2 DRAM, with details in Table II. Off-chip memory is modeled using Ramulator2 [25] configured with an HBM2 at 2 Gbps. All hardware components are simulated on a cycle-by-cycle basis to ensure timing accuracy.

Baselines. We compare to two state-of-the-art accelerators.

TABLE II: SegFold Hardware Configuration

Parameter	Value
PE array	16×16
Per-row hardware ($\times$16 rows)	shifter, spad bank, LUT bank
Active B window size	32
Cache size, associativity, line size	1.5 MiB, 16-way, 128 B
DRAM	HBM2-8Gb, 2 Gbps

Flexagon [28] is a reconfigurable accelerator capable of supporting multiple canonical dataflows. The on-chip components of Flexagon are modeled using the open-source `STONNE` simulator [30], integrated with the same Ramulator-based memory backend for consistency. Since Flexagon was originally designed as a 1D array accelerator, it is extended to a 2D configuration for fair comparison. This extension duplicates the on-chip cache while maintaining a shared off-chip DRAM. To match the compute resources of SegFold, Flexagon's 1D array of 128 PEs is scaled into a 2D array of 2×128 PEs. On-chip bandwidth is preserved by scaling both the reduction network and distribution network to 128 elements per cycle for each 1D array. The original Flexagon network remains unchanged along the second dimension; the 2D extension is achieved by tiling along M to distribute the workload evenly across the two PE arrays.

Spada [24] is a runtime-adaptive SpGEMM accelerator that dynamically adjusts its window to the sparsity of matrix A. We use its open-source simulator unchanged as our baseline. For the non-square evaluation (Fig. 9(a)), we additionally integrate Spada with the same Ramulator2 HBM2 memory backend used by SegFold to ensure a fair memory-system comparison.

Area and Energy. We employ the ASAP7 7nm standard-cell library as the target technology for RTL synthesis [2]. We use Design Compiler to elaborate all RTL sources and invoke `compile_ultra` for timing-driven optimization. We report the post-synthesis timing, area, resources, and power.

Workloads. We select fifteen matrices from SuiteSparse [8] as our baseline benchmark suite for the overall and non-square comparisons, covering a range of application domains, matrix sizes, aspect ratios, and sparsity levels. The ablation studies in §VI-C draw from a slightly different subset to expose mechanism-specific behavior. Throughout, we report *density* as $\mathrm{nnz}/(M \times N)$. These matrices are characterized in Table III. Across all experiments, the matrices span dimensions from hundreds to tens of thousands of rows and densities across more than two orders of magnitude. The test set includes both square and non-square matrices to capture the shape impact of matrix multiplication. The benchmark suite exercises a representative range of irregular access patterns. Unless otherwise specified, we use the transpose of the matrix as B for matrix multiplication.

Tiling. Tiling is applied along the Cartesian dimensions M and N. Tile sizes are statically determined based on the distribution of nonzero values in the corresponding C tiles. To accommodate the tiling, when a virtual row of C exceeds the physical PE-row capacity, overflow values are spilled to the per-row spad. Because the spad is sized to accommodate

TABLE III: SuiteSparse matrices used in our evaluation.

Matrix	M	N	Density	Application domain
fv1	9604	9064	9.79e−4	2D/3D problem
flowmeter0	9669	9669	7.21e−4	Model reduction
delaunay_n13	8192	8192	7.32e−4	Undirected graph
ca-GrQc	5242	5242	1.05e−3	Undirected graph
ca-CondMat	23133	23133	3.49e−4	Undirected graph
poisson3Da	13514	13514	1.93e−3	CFD
bcspwr06	1454	1454	2.51e−3	Power network
tols4000	4000	4000	5.49e−4	CFD
rdb5000	5000	5000	1.18e−3	CFD
gemat1	4929	10595	8.92e−4	Power network
lp_woodw	1098	8418	4.06e−3	Linear programming
pcb3000	3960	7732	1.88e−3	Circuit simulation
Franz6	7576	3016	1.99e−3	Combinatorial problem
Franz8	16728	7176	8.36e−4	Combinatorial problem
psse1	14318	11028	3.63e−4	Power network

the expected maximum overflow, which is determined by the tile dimensions along M and N and the anticipated density of C, spills are infrequent under our default tiling configuration.

VI. Evaluation

We evaluate SegFold's end-to-end performance against Spada and Flexagon, perform a per-component ablation, study sensitivity to key hardware parameters and input sparse patterns, and report post-synthesis area and power.

A. Overall Performance

Fig. 8 shows that SegFold achieves a $1.95\times$ geometric-mean speedup over Spada and a $5.3\times$ geometric-mean speedup over the best-performing Flexagon configuration across all workloads. The much larger $5.3\times$ speedup over Flexagon reflects the limitation of static dataflows: Flexagon must commit to a single dataflow—inner-product, outer-product, or Gustavson—per tile, and even its best per-tile choice cannot simultaneously exploit reuse on all three operands. SegFold's dynamic scheduling sidesteps this by reordering work within each tile based on the runtime nonzero pattern.

On highly sparse matrices with structured, non-uniform nonzero distributions, SegFold achieves $1.08\times$ to $5.75\times$ speedups over Spada. The advantage comes from SegFold's two core dynamic mechanisms operating *within* a tile: (1) SELECTA dynamically reorders (m, k) pairs in the active window to maximize B-row reuse and avoid C-row contention, and (2) SEGMENTBC dynamically remaps partial sums across PEs based on the evolving $\mathcal{V}$ space state. Spada, in contrast, adapts only its window *size* at tile granularity while keeping the scheduling within each window static, leaving sub-tile reuse and load-balance opportunities unexploited. This gap is amplified on highly sparse matrices, where SegFold's dynamic scheduling can skip whole empty regions of the iteration space outright (e.g., k columns with no A/B-nonzeros), while Spada must still follow its static loop over the entire window even when most of it contributes no work. The one exception is ca-GrQc, on which SegFold underperforms Spada ($0.59\times$): its scale-free graph structure produces a few extremely dense rows that overwhelm SegFold's per-row PE allocation, while Spada's tile-level adaptation handles them better.

B. Non-square Performance

Figure 9(a) compares SegFold against Spada on non-square SuiteSparse matrices. These non-square comparisons are done by multiplying each matrix by its own transpose. SegFold outperforms Spada on tall matrices, achieving $1.42\times$ geomean speedup over Spada, where its dynamic dataflow effectively exploits the elongated row structure. On wide matrices, however, SegFold falls behind: two out of three tested matrices underperform Spada. The cause is how SegFold handles the K dimension: because we do not tile along K within a tile, a large K relative to M creates load imbalance across A rows. Transposing the matrices does not help here because we are already computing $A \times A^\top$ (a self-transpose multiply). However, there are cases where transposition could provide critical optimization.

As shown in Fig. 9(b), Direction 1 computes $A^{M,K}_{\text{real}} \times S^{K,N}$, where $M \neq K$ and $K = N$ while Direction 2 computes $S^{M,K} \times \left(A^{N,K}_{\text{real}}\right)^\top$, where $M = K$ and $K \neq N$. They are arithmetically equivalent up to an output transpose, effectively swapping which operand drives SELECTA. Transposing the wide matrices recovers 2.4–$3.0\times$ over Direction 1, confirming that the M/K ratio has a significant impact on SegFold's performance: when an operand has its reduction-side dimension K much larger than its output-side dimension, making it the second operand places its short axis along the multiplication's output dimension N. The dataflow then iterates along the short N rather than the long K, improving the efficiency of SELECTA's scan. Conversely, on tall matrices Direction 1 is already favorable. Selecting the multiplication direction can often be decided in advance, yielding several-fold speedups.

C. Ablation Studies

We isolate the effects of two components of SegFold: (i) the dynamic scheduling and (ii) the dynamic mapping.

1) Effect of Dynamic Scheduling: To understand the impact of SegFold's dynamic dataflow, we perform an experiment with fixed k iteration order, making the dataflow resemble a constrained outer-product scheme: instead of dynamically reordering k within the active window, we process k in a predetermined sequence. The final result shows this reduces normalized performance to 0.670 ± 0.065 of the baseline, indicating that dynamic k reordering is important for exposing segment-level parallelism and keeping PEs busy.

2) Effect of Dynamic Mapping: To isolate the effect of dynamic mapping, we compare SegFold's LUT-based mapping against two alternatives, both using the same dataflow and hardware resources but with different mapping logic:

1) **Zero-Offset mapping**: the head of the B row is always mapped to the beginning of the PE row ($f_{t_{\text{in}}}(m, n) = 0$).
2) **Ideal-Network mapping**: an oracle mapper that always finds the optimal placement with no stale-index overhead.

Figure 10 reports the speedup of each mapping method across 16 SuiteSparse matrices, normalized to the zero-offset baseline. SegFold's LUT-based mapper achieves a geometric-mean speedup of $1.20\times$ over the zero-offset policy. Matrices

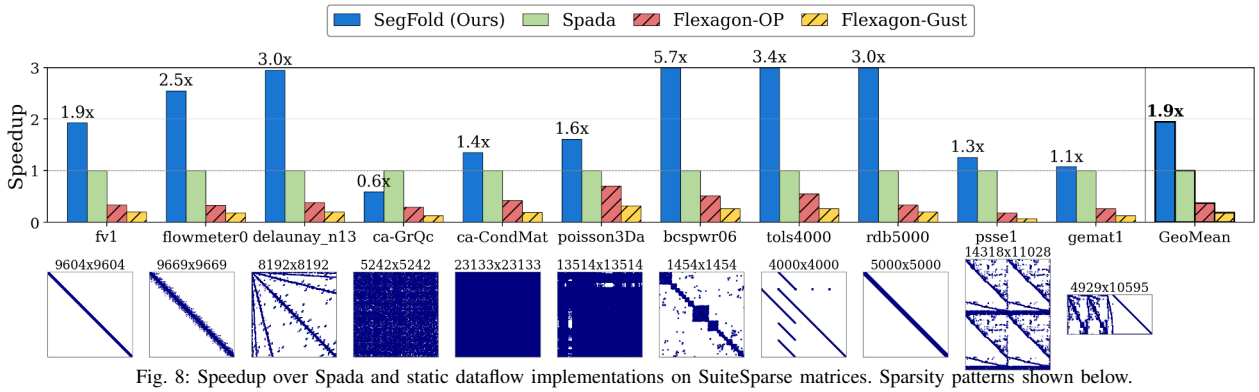


Fig. 8: Speedup over Spada and static dataflow implementations on SuiteSparse matrices. Sparsity patterns shown below.

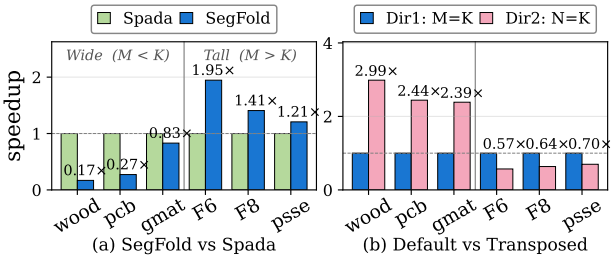


Fig. 9: Nonsquare SuiteSparse matrices. **(a)** SegFold vs. Spada speedup, normalized to Spada. **(b)** Effect of multiplication direction on SegFold throughput, normalized to nonsquare matrix as A.

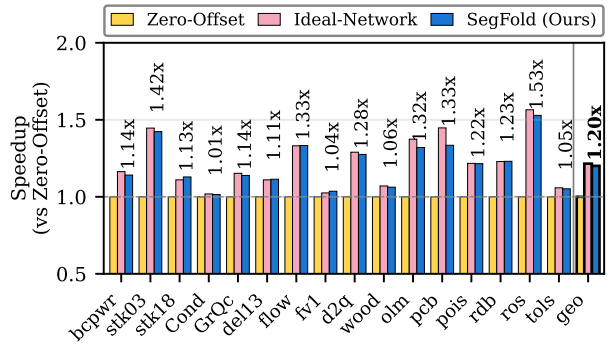


Fig. 10: Speedup of different mapping methods (Zero-Offset, Ideal-Network, SegFold) on SuiteSparse matrices, normalized to Zero-Offset.

with more irregular sparsity patterns (e.g., pcb3000, olm5000, flowmeter0) benefit most from dynamic scheduling, as the LUT-based mapper adapts to instantaneous PE occupancy and avoids long segment traversals. Compared to the ideal oracle mapping, SegFold's LUT-based scheduling incurs only a 1.2% average overhead, demonstrating that our hardware approximation closely tracks the theoretical optimum.

Attribution Summary. To quantify the contribution of each dynamic mechanism in both the dataflow and microarchitecture, we conduct an incremental ablation study across 12 SuiteSparse matrices. Figure 11 shows the performance breakdown of SegFold, achieving a geometric-mean speedup of 3.1× over the base configuration. Across different sparsity patterns, each dynamic mechanism contributes to the overall speedup, with SELECTA delivering the largest gain—

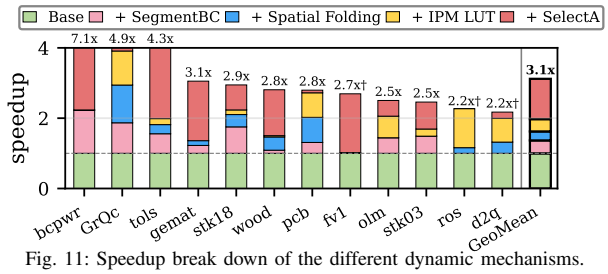


Fig. 11: Speedup break down of the different dynamic mechanisms.

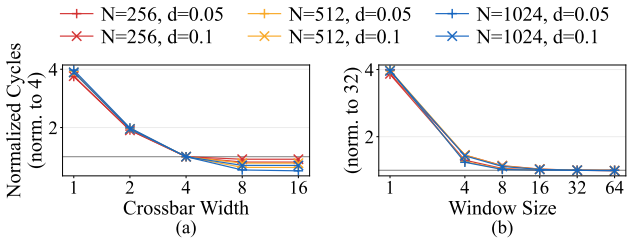


Fig. 12: Hardware-parameter sensitivity studies across three matrix sizes ($N = 256, 512, 1024$) and two density levels ($d = 0.05, 0.1$). **(a)** Crossbar Width sweep, normalized to BRL= 4. **(b)** Window Size sweep, normalized to W= 32. Color encodes matrix size, marker encodes density (+: $d = 0.05$, ×: $d = 0.1$).

indicating that dynamic scheduling plays a critical role in handling highly irregular sparse matrices. SEGMENTBC, spatial folding, and the IPM LUT provide additional gains, with their relative benefits depending on the matrix's structural properties and output sparsity patterns.

D. Sensitivity Studies

We study SegFold's sensitivity along two axes: *hardware parameters* and *input characteristics*.

1) Hardware Parameter Sensitivity: To understand how hardware parameters shape SegFold's performance, we perform two sensitivity studies across three matrix sizes $\{256, 512, 1024\}$ and two density levels $\{0.05, 0.1\}$: (i) varying the bandwidth of the vector multicast network that delivers B rows, and (ii) varying the active-window size.

Vector Multicast Bandwidth. As shown in Fig. 5(a), SegFold's global network routes B rows from memory to PE rows through a vector multicast network. To observe sensitivity, we vary the network multicast width from 1 to 16 B rows

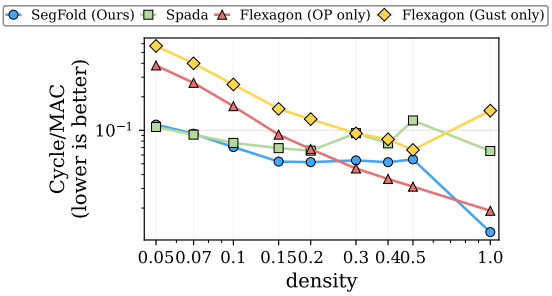


Fig. 13: Per-simulation efficiency (cycles per MAC, lower is better) on synthetic square matrices across densities, comparing Flexagon (OP and Gustavson), Spada, and SegFold.

per cycle, keeping all other parameters fixed. As shown in Fig. 12, performance improves noticeably from 1 to 4 rows per cycle across all matrix sizes and densities, but the marginal benefit beyond 4 rows per cycle quickly diminishes. At higher density ($d = 0.1$), the sensitivity is more pronounced for larger matrices, as increased nonzero density creates more contention on the network. Based on this trend and the associated area/wiring cost, we choose a bandwidth of 4 B rows per cycle.

Window Size of Active B Rows. As discussed in §IV-B, the active window over B rows controls how many k values and B rows can be reordered and scheduled in parallel. We sweep window sizes from 1 to 64, and report the normalized cycles in Fig. 12. Performance improves substantially as the window grows up to 32, but shows little further gain beyond 32, consistently across both density levels. At higher density ($d = 0.1$), the benefit of increasing window size is slightly more pronounced, as denser matrices expose more reordering opportunities. We therefore adopt a window size of 32 as our default configuration, as it offers a good trade-off between benefit and metadata/storage overhead.

2) Input Sensitivity: To understand how input characteristics affect SegFold's performance, we perform two sensitivity studies with the synthetic matrices: (i) varying the density of the two input matrices from highly sparse to entirely dense; and (ii) varying the relative sparsity of the two input matrices to create asymmetric sparsity patterns.

Density Sweep. We test synthetic matrices of sizes 256 to 1024 with densities ranging from 0.05 to 1.0. Figure 13 reports the average cycles per MAC across the three accelerators. As density increases, SegFold's per-MAC efficiency stays roughly flat through mid densities and then drops sharply at the fully dense end, while Spada's degrades sharply once density exceeds 0.4. This transition reflects bandwidth saturation in Spada's row-sequential 16-channel memory. Flexagon's outer-product (OP) dataflow shows the biggest improvement with increasing density, as its static dataflow can better exploit reuse opportunities when more nonzeros are present, and the fixed control overhead is amortized. This also explains why SegFold's speedup over Flexagon is relatively large on the SuiteSparse matrices, which are generally sparser than the synthetic ones. Notably, in the fully dense case, SegFold outperforms all the baselines: its 2D array natively supports

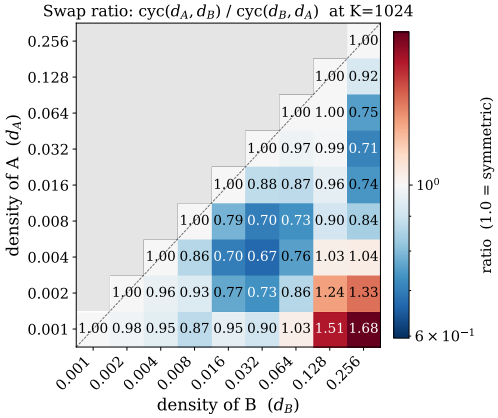


Fig. 14: Asymmetric-sparsity sensitivity at $K = 1024$.

dense GEMM, and the dynamic-mapping overhead is diminished when the matrix is dense, since the $\mathcal{V}$ space mapping is essentially static—every position is occupied, leaving no shifts or skip decisions for the merge network to make, while baselines have the static overhead introduced by isolating the multiplication and reduction phases. For sparsity levels beyond the figure's range, SegFold's speedup over Spada increases.

Asymmetric Sparsity. SegFold treats A and B asymmetrically, raising the question of whether sparsity differences between A and B influence whether $A \times B$ or $A^\top \times B^\top$ is faster. To understand this, we sweep (d_A, d_B) pairs over synthetic matrices with size 1024 and report the swap ratio $\mathrm{cyc}(d_A, d_B)/\mathrm{cyc}(d_B, d_A)$ in Fig. 14: ratios < 1 (blue) favor placing the sparser matrix as operand A, while ratios > 1 (red) favor placing the denser one there. Since the upper triangle is the reciprocal of the lower, we focus on the lower-right half ($d_A \leq d_B$). Most of this region is blue: having the sparser matrix as operand A is faster because SELECTA's fine-grained scheduling on A leverages high sparsity better than the coarse-grained loader on B can.

However, in the red corner where the disparity in density between A and B becomes very large, having the denser matrix as the first operand becomes faster. When A's sparsity is extreme, a substantial fraction of A's rows contain no nonzeros, yet each row still triggers a SELECTA iteration with its associated scheduling and pipeline overhead; the fine-grained selection that benefits sparse A in the common case now operates over many empty rows and produces no useful work. Swapping the operands places the denser matrix in operand A—every row is occupied, so no iteration is wasted—while the very sparse matrix moves to operand B, where the coarse-grained, demand-driven loader simply skips empty rows at no cost. The crossover from blue to red occurs once the density ratio d_B/d_A grows large enough (in our experiments, around $32\times$ to $64\times$) that the per-row SELECTA overhead saved by the swap exceeds the fine-grained sparsity benefit lost on A.

E. RTL Results

Table IV reports the post-synthesis RTL estimates of power and area for SegFold. We synthesize all modules using the

TABLE IV: Power and area of SegFold. Left: ASAP7 7 nm synthesis at 1 GHz. Right: estimated 28 nm scaling at 1 GHz (with pipeline modifications).

Component	ASAP7 7 nm		Est. 28 nm	
	Area [μm^2]	Power [mW]	Area [μm^2]	Power [mW]
PE ($\times$256)	26,304	59.1	$\sim$231,400	$\sim$342
Switch ($\times$256)	10,967	27.1	$\sim$87,700	$\sim$156
FIFO Buffer ($\times$256)	105,600	263.4	$\sim$887,000	$\sim$1,574
Scratchpad ($\times$16)	16,025	34.3	$\sim$134,600	$\sim$208
Mem Controller ($\times$1)	1,603	1.8	$\sim$13,470	$\sim$11
Total	**160,499** ($0.160\,\text{mm}^2$)	**385.7**	**$\sim$1,354,200** ($\sim 1.35\,\text{mm}^2$)	**$\sim$2,290**

ASAP7 7 nm standard-cell library [2] at 1 GHz. For the 2D mesh, we synthesize a single PE, switch, and FIFO buffer, and scale their results to a 16×16 grid. Each PE row includes a spad for overflow C entries; these are synthesized per bank and scaled by 16 rows. The global memory controller is synthesized separately. At 7 nm, the total design occupies $0.160\,\text{mm}^2$ and consumes 385.7 mW. To enable comparison with prior work synthesized at older technology nodes, we also provide estimated 28 nm numbers by applying standard area ($\sim 8\times$) and power ($\sim 5.5\times$) scaling factors, with additional pipeline stages inserted to meet 1 GHz timing at 28 nm.

For comparison, Flexagon [28] reports a total area (without cache) of $1.35\,\text{mm}^2$ and power consumption of 856 mW at 28 nm. When scaled to the same 28 nm technology node, SegFold's estimated area of $\sim 1.35\,\text{mm}^2$ is comparable, while providing the additional hardware for dynamic scheduling (merge network, scratchpads) that enables the performance gains shown above. The estimated 28 nm power of $\sim$2.3 W assumes conservative vectorless 50% switching activity; real sparse workloads exhibit lower activity factors, which would substantially reduce dynamic power.

VII. Additional Related Work

Reconfigurable Architectures. Reconfigurable architectures [13], [35], [37] are inherently programmable and can support different dataflows. Prior architectures have explored specialization for sparsity, such as SPU [5] for sparse inner-product joins, and Capstan [18], [40] for parallel indirect memory access in Gustavson. To our knowledge, dynamic dataflow optimizations have not been explored on a reconfigurable architecture, and they would likely be costly given coarse-grain reconfigurable primitives.

Dynamic Prediction. Dynaflow [48] predicts the best dataflow for dataflow-flexible accelerators like those above, using ML techniques trained on dataset sparsity patterns. SparseAdapt [34] is a runtime system for a flexible reconfigurable design called Transmuter [35], which can change microarchitectural policies depending on sparse data patterns.

Tile-Ordering Strategies. Inter-tile ordering strategies such as Hilbert or Z-order traversals—used in CUBE [52] and Mosaic [27]—and the sliding-window tiling in Spahet [19] optimize the *order* in which tiles are visited to improve cross-tile cache locality, but leave the scheduling of work *within* a tile static. SegFold's dynamic scheduling is orthogonal: it operates at sub-tile granularity, reordering operations inside a tile based on runtime sparsity, and could therefore compose with any of these inter-tile ordering schemes.

Flexibility and Dynamism in Related Domains. EIE [15] is a SpMV accelerator that mitigates load imbalance with large input queues. Cerebrus [20] is a SpMV accelerator that can support multiple dataflow patterns. ZeNa [22] is a sparse CNN accelerator that uses work-stealing queues to perform load balancing at a tile level. Graph processing accelerators face similar sparsity patterns and encounter the same load-balance and reuse challenges. PolyGraph [4] uses offline preprocessing, which aims to improve both load balance and locality. HATS [29] uses bounded depth-first search ordering to improve locality. AWB-GCN [12] is a graph convolution accelerator, which dynamically load balances rows among PEs, and further splits pathologically long rows. These graph accelerators all exploit locality in settings where the output structure is known ahead of time; SegFold faces the additional challenge of discovering the output nonzero pattern on-the-fly during SpGEMM execution, requiring runtime locality decisions rather than offline preprocessing.

Comparison with Preprocessing-Based Approaches. Building on the Gustavson-style preprocessing designs introduced in §II, we contrast SegFold's runtime approach with offline preprocessing in more detail. Gamma [51] and Zed [6] use offline preprocessing to reorder rows of the stationary matrix, grouping rows with similar sparsity patterns to improve streaming-matrix reuse under a Gustavson dataflow. This preprocessing must be performed once per matrix and amortized across repeated SpGEMM invocations. In contrast, SegFold achieves similar reuse benefits—grouping A columns with overlapping nonzero patterns via SELECTA—entirely at runtime, with no preprocessing step. This makes SegFold particularly advantageous for workloads where the sparse matrices change frequently (e.g., dynamic graphs or iterative solvers), as the cost of preprocessing cannot be amortized. For static matrices used repeatedly, preprocessing-based approaches may offer complementary benefits, and combining offline reordering with SegFold's runtime scheduling is a promising direction for future work.

VIII. Conclusion

This work introduces Segment, a novel dynamic dataflow for SpGEMM that simultaneously optimizes data reuse and load balance, overcoming limitations of conventional static dataflows. Through the co-designed SegFold accelerator, we demonstrate that fine-grained dynamic scheduling and remapping of work across PEs can significantly improve performance and utilization across diverse sparsity patterns and matrix sizes. Our results—a $1.95\times$ geometric-mean speedup over the best dynamic baseline and $5.3\times$ over the best static baseline—highlight the value of incorporating dynamism into sparse-matrix accelerators, providing a new pathway for efficient execution of irregular workloads.

REFERENCES

[1] A. Buluç and J. R. Gilbert, "On the representation and multiplication of hypersparse matrices," in *2008 IEEE International Symposium on Parallel and Distributed Processing (IPDPS)*, 2008, pp. 1–11.

[2] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm FinFET predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.

[3] N. Corp, "NVIDIA GPU programming guide, v2.2.1, November, 2004."

[4] V. Dadu, S. Liu, and T. Nowatzki, "PolyGraph: Exposing the value of flexibility for graph processing accelerators," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 595–608.

[5] V. Dadu, J. Weng, S. Liu, and T. Nowatzki, "Towards general purpose acceleration by exploiting common data-dependence forms," ser. MICRO '52. New York, NY, USA: ACM, 2019, pp. 924–939. [Online]. Available: http://doi.acm.org/10.1145/3352460.3358276

[6] P. Dangi, Z. Bai, R. Juneja, D. Wijerathne, and T. Mitra, "ZeD: A generalized accelerator for variably sparse matrix computations in ML," in *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 246–257. [Online]. Available: https://doi.org/10.1145/3656019.3689905

[7] S. Dave, R. Baghdadi, T. Nowatzki, S. Avancha, A. Shrivastava, and B. Li, "Hardware acceleration of sparse and irregular tensor computations of ML models: A survey and insights," *Proceedings of the IEEE*, vol. 109, no. 10, pp. 1706–1752, 2021.

[8] T. A. Davis and Y. Hu, "The university of florida sparse matrix collection," *ACM Transactions on Mathematical Software*, vol. 38, no. 1, pp. 1:1–1:25, 2011.

[9] J. Dean, "Recent advances in artificial intelligence via machine learning and the implications for computer system design," in *2017 IEEE Hot Chips 29 Symposium*, 2017.

[10] J. Gao, W. Ji, F. Chang, S. Han, B. Wei, Z. Liu, and Y. Wang, "A systematic survey of general sparse Matrix-Matrix multiplication," *ACM Comput. Surv.*, vol. 55, no. 12, Mar. 2023. [Online]. Available: https://doi.org/10.1145/3571157

[11] H. N. Genc, H. Kim, P. Ganesh, and Y. S. Shao, "Stellar: An automated design framework for dense and sparse spatial accelerators," in *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '24. IEEE Press, 2024, p. 409–422. [Online]. Available: https://doi.org/10.1109/MICRO61859.2024.00038

[12] T. Geng, A. Li, R. Shi, C. Wu, T. Wang, Y. Li, P. Haghi, A. Tumeo, S. Che, S. Reinhardt, and M. C. Herbordt, "AWB-GCN: A graph convolutional network accelerator with runtime workload rebalancing," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 922–936.

[13] G. Gobieski, S. Ghosh, M. Heule, T. Mowry, T. Nowatzki, N. Beckmann, and B. Lucia, "RipTide: A programmable, energy-minimal dataflow compiler and architecture," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 546–564.

[14] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 151–165. [Online]. Available: https://doi.org/10.1145/3352460.3358291

[15] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: efficient inference engine on compressed deep neural network," in *Proceedings of the 43rd International Symposium on Computer Architecture*, ser. ISCA '16. IEEE Press, 2016, p. 243–254. [Online]. Available: https://doi.org/10.1109/ISCA.2016.30

[16] X. He, S. Pal, A. Amarnath, S. Feng, D.-H. Park, A. Rovinski, H. Ye, Y. Chen, R. Dreslinski, and T. Mudge, "Sparse-TPU: Adapting systolic arrays for sparse matrices," in *Proceedings of the 34th ACM international conference on supercomputing*, 2020, pp. 1–12.

[17] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher, "ExTensor: An accelerator for sparse tensor algebra," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-52. New York, NY, USA: Association for Computing Machinery, 2019, p. 319–333. [Online]. Available: https://doi.org/10.1145/3352460.3358275

[18] O. Hsu, A. Rucker, T. Zhao, V. Desai, K. Olukotun, and F. Kjolstad, "Stardust: Compiling sparse tensor algebra to a reconfigurable dataflow architecture," in *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, ser. CGO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 628–643. [Online]. Available: https://doi.org/10.1145/3696443.3708918

[19] H. Huang, P. Yao, Z. An, Y. Sun, A. Hu, P. Xu, L. Zheng, X. Liao, and H. Jin, "SpaHet: A software/hardware co-design for accelerating heterogeneous-sparsity based sparse matrix multiplication," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3649329.3655944

[20] S. Hwang, D. Baek, J. Park, and J. Huh, "Cerberus: Triple mode acceleration of sparse matrix and vector multiplication," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 2, May 2024. [Online]. Available: https://doi.org/10.1145/3653020

[21] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-l. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ser. ISCA '17. New York, NY, USA: ACM, 2017, pp. 1–12. [Online]. Available: http://doi.acm.org/10.1145/3079856.3080246

[22] D. Kim, J. Ahn, and S. Yoo, "Zena: Zero-aware neural network accelerator," *IEEE Design & Test*, vol. 35, no. 1, pp. 39–46, 2017.

[23] K. Koul, M. Strange, J. Melchert, A. Carsello, Y. Mei, O. Hsu, T. Kong, P.-H. Chen, H. Ke, K. Zhang *et al.*, "Onyx: A programmable accelerator for sparse tensor algebra," in *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE Computer Society, 2024, pp. 1–91.

[24] Z. Li, J. Li, T. Chen, D. Niu, H. Zheng, Y. Xie, and M. Gao, "Spada: Accelerating sparse matrix multiplication with adaptive dataflow," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 747–761. [Online]. Available: https://doi.org/10.1145/3575693.3575706

[25] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible DRAM simulator," 2023. [Online]. Available: https://arxiv.org/abs/2308.11030

[26] S. Luo, B. Wang, Y. Shi, X. Zhang, Q. Xue, and S. Ma, "Sparm: A sparse matrix multiplication accelerator supporting multiple dataflows," in *2024 IEEE 35th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2024, pp. 122–130.

[27] S. Maass, C. Min, S. Kashyap, W. Kang, M. Kumar, and T. Kim, "Mosaic: Processing a trillion-edge graph on a single machine," in *Proceedings of the Twelfth European Conference on Computer Systems*, ser. EuroSys '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 527–543. [Online]. Available: https://doi.org/10.1145/3064176.3064191

[28] F. Muñoz Martínez, R. Garg, M. Pellauer, J. L. Abellán, M. E. Acacio, and T. Krishna, "Flexagon: A multi-dataflow sparse-sparse matrix multiplication accelerator for efficient DNN processing," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 252–265. [Online]. Available: https://doi.org/10.1145/3582016.3582069

[29] A. Mukkara, N. Beckmann, M. Abeydeera, X. Ma, and D. Sanchez, "Exploiting locality in graph analytics through hardware-accelerated traversal scheduling," in *MICRO*. IEEE, 2018, pp. 1–14.

[30] F. Muñoz-Martínez, J. L. Abellán, M. E. Acacio, and T. Krishna, "STONNE: Enabling cycle-level microarchitectural simulation for DNN

inference accelerators," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*, 2021, pp. 201–213.

[31] N. Nayak, T. O. Odemuyiwa, S. Ugare, C. Fletcher, M. Pellauer, and J. Emer, "TeAAL: A declarative framework for modeling sparse tensor accelerators," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1255–1270. [Online]. Available: https://doi.org/10.1145/3613424.3623791

[32] E. Nurvitadhi, A. Mishra, and D. Marr, "A sparse matrix vector multiply accelerator for support vector machine," in *2015 International Conference on Compilers, Architecture and Synthesis for Embedded Systems (CASES)*, Oct 2015, pp. 109–116.

[33] S. Pal, J. Beaumont, D. Park, A. Amarnath, S. Feng, C. Chakrabarti, H. Kim, D. Blaauw, T. Mudge, and R. Dreslinski, "OuterSPACE: An outer product based sparse matrix multiplication accelerator," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2018, pp. 724–736.

[34] S. Pal, A. Amarnath, S. Feng, M. O'Boyle, R. Dreslinski, and C. Dubach, "SparseAdapt: Runtime control for sparse linear algebra on a reconfigurable accelerator," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1005–1021. [Online]. Available: https://doi.org/10.1145/3466752.3480134

[35] S. Pal, S. Feng, D.-h. Park, S. Kim, A. Amarnath, C.-S. Yang, X. He, J. Beaumont, K. May, Y. Xiong, K. Kaszyk, J. M. Morton, J. Sun, M. O'Boyle, M. Cole, C. Chakrabarti, D. Blaauw, H.-S. Kim, T. Mudge, and R. Dreslinski, "Transmuter: Bridging the efficiency gap using memory and dataflow reconfiguration," in *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 175–190. [Online]. Available: https://doi.org/10.1145/3410463.3414627

[36] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ser. ISCA '17. New York, NY, USA: ACM, 2017, pp. 27–40. [Online]. Available: http://doi.acm.org/10.1145/3079856.3080254

[37] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel paterns," ser. ISCA '17. New York, NY, USA: ACM, 2017, pp. 389–402. [Online]. Available: http://doi.acm.org/10.1145/3079856.3080256

[38] E. Qin, G. Jeong, W. Won, S.-C. Kao, H. Kwon, S. Srinivasan, D. Das, G. E. Moon, S. Rajamanickam, and T. Krishna, "Extending sparse tensor accelerators to support multiple compression formats," in *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2021, pp. 1014–1024.

[39] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 58–70.

[40] A. Rucker, M. Vilim, T. Zhao, Y. Zhang, R. Prabhakar, and K. Olukotun, "Capstan: A vector RDA for sparsity," 2021.

[41] L. Song, Y. Chi, L. Guo, and J. Cong, "Serpens: A high bandwidth memory based accelerator for general-purpose sparse matrix-vector multiplication," in *Proceedings of the 59th ACM/IEEE design automation conference*, 2022, pp. 211–216.

[42] L. Song, Y. Chi, A. Sohrabizadeh, Y.-k. Choi, J. Lau, and J. Cong, "Sextans: A streaming accelerator for general-purpose sparse-matrix dense-matrix multiplication," in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2022, pp. 65–77.

[43] N. Srivastava, H. Jin, J. Liu, D. Albonesi, and Z. Zhang, "MatRaptor: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 766–780.

[44] N. Srivastava, H. Jin, S. Smith, H. Rong, D. Albonesi, and Z. Zhang, "Tensaurus: A versatile accelerator for mixed sparse-dense tensor computations," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 689–702.

[45] B. Wang, S. Ma, S. Luo, L. Wu, J. Zhang, C. Zhang, and T. Li, "SparGD: A sparse GEMM accelerator with dynamic dataflow," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 2, Jan. 2024. [Online]. Available: https://doi.org/10.1145/3634703

[46] B. Wang, S. Ma, Y. Zhao, S. Luo, L. Wu, J. Zhang, D. Li, T. Li, and Z. Chen, "SpMARD: A sparse-sparse matrix multiplication accelerator with reconfigurable dataflow for DNN workloads," *ACM Trans. Archit. Code Optim.*, Aug. 2025, just Accepted. [Online]. Available: https://doi.org/10.1145/3747847

[47] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer, "Sparseloop: An analytical approach to sparse tensor accelerator modeling," in *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '22. IEEE Press, 2023, p. 1377–1395. [Online]. Available: https://doi.org/10.1109/MICRO56248.2022.00096

[48] S. Yadav and B. Asgari, "DynaFlow: An ML framework for dynamic dataflow selection in SpGEMM accelerators," *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 189–192, 2025.

[49] Y. Yang, J. S. Emer, and D. Sanchez, "Trapezoid: A versatile accelerator for dense and sparse matrix multiplications," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 931–945.

[50] P. Yao, L. Zheng, Y. Huang, Q. Wang, C. Gui, Z. Zeng, X. Liao, H. Jin, and J. Xue, "ScalaGraph: A scalable accelerator for massively parallel graph processing," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 199–212.

[51] G. Zhang, N. Attaluri, J. S. Emer, and D. Sanchez, "Gamma: Leveraging Gustavson's algorithm to accelerate sparse matrix multiplication," ser. ASPLOS 2021. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3445814.3446702

[52] M. Zhang, Y. Wu, K. Chen, X. Qian, X. Li, and W. Zheng, "Exploring the hidden dimension in graph processing," in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'16. USA: USENIX Association, 2016, p. 285–300.

[53] S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, and Y. Chen, "Cambricon-X: An accelerator for sparse neural networks," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2016, pp. 1–12.

[54] Z. Zhang, H. Wang, S. Han, and W. J. Dally, "Sparch: Efficient architecture for sparse matrix multiplication," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 261–274.

[55] K. Zhong, Z. Zhu, G. Dai, H. Wang, X. Yang, H. Zhang, J. Si, Q. Mao, S. Zeng, K. Hong *et al.*, "Feasta: A flexible and efficient accelerator for sparse tensor algebra in machine learning," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 349–366.

[56] X. Zhou, Z. Du, Q. Guo, S. Liu, C. Liu, C. Wang, X. Zhou, L. Li, T. Chen, and Y. Chen, "Cambricon-S: Addressing irregularity in sparse neural networks through a cooperative software/hardware approach," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2018, pp. 15–28.

APPENDIX

Abstract

We release the complete SegFold artifact, consisting of a cycle-accurate C++ simulator, real-world and synthetic benchmark suites, RTL source with synthesis reports, and automated scripts that regenerate every figure and table in the paper. Concretely, the artifact contains: (i) `csegfold`, a simulator that faithfully models SegFold's microarchitecture and dynamic dataflow, (ii) a curated set of sparse matrices drawn from the SuiteSparse collection, (iii) SystemVerilog RTL and corresponding area/power/timing reports for all hardware modules, and (iv) end-to-end automation that builds the simulator, runs all experiments, and produces publication-ready plots.

Artifact Check-List (Meta-Information)

- **Algorithm:** Segment dataflow for SpGEMM with fine-grained dynamic scheduling and work remapping.
- **Program:** `csegfold` — cycle-accurate C++ simulator.
- **Compilation:** CMake $\geq$ 3.15, GCC 10+ (C++20 required). Ramulator2 is pulled automatically during the build via CMake FetchContent.
- **Binary:** `csegfold/build/csegfold`, compiled from source.
- **Data set:** $\sim$20 SuiteSparse matrices ($\sim$50 MB, auto-downloaded), covering the 15-matrix baseline suite plus the additional matrices used in ablation studies; synthetic matrices are created on-the-fly.
- **Hardware:** Commodity server — $\geq$4 CPU cores and $\geq$64 GB RAM (16+ cores and 256 GB RAM recommended for full parallelism).
- **Run-time environment:** Linux (tested on Ubuntu 22.04+), Python $\geq$ 3.8. A Docker image is also available.
- **Metrics:** Simulated cycle counts and speedup relative to prior accelerators.
- **Output:** Per-experiment CSV files and PDF/PNG plots corresponding to Figures 8–12.
- **Experiments:** 209 individual simulation runs, completing in $\sim$2 hours on a 16-core machine.
- **How much disk space required?** $\sim$2 GB (source, benchmarks, and generated outputs).
- **How much time is needed to prepare the workflow?** $\sim$5 minutes for building the simulator and downloading matrices.
- **How much time is needed to complete experiments?** $\sim$2 hours at 16 cores; runtime scales roughly linearly with core count.
- **Publicly available?** Yes.
- **Code licenses?** MIT License.
- **Archived?** Yes (GitHub + Zenodo) DOI: 10.5281/zenodo.19453259.

Description

How to Access: The artifact is hosted on GitHub:
https://github.com/PolyArch/SegFold-AE

and also Zenodo:
https://doi.org/10.5281/zenodo.19453259

Hardware Dependencies:

- CPU: 4 cores minimum, 16+ cores recommended.
- RAM: 64 GB minimum, 256 GB recommended. Memory-intensive experiments (breakdown and mapping ablation) may consume up to 50 GB per process.
- Disk: 2–5 GB.

Software Dependencies:

- OS: Ubuntu 22.04 or later (other Linux distributions are expected to work).
- Toolchain: GCC 10+ (C++20), CMake $\geq$ 3.15.
- Python $\geq$ 3.8 with numpy, scipy, matplotlib, pandas, and pyyaml.
- Docker (optional, for a self-contained environment).

Data Sets: The experiments use 20 sparse matrices from the SuiteSparse Matrix Collection (https://sparse.tamu.edu/), fetched automatically by a provided download script. Synthetic matrices for sensitivity studies are generated at runtime.

Installation

Native build.

1) Build the simulator and run a smoke test:

```
./scripts/setup.sh
```

2) Download benchmark matrices:

```
python3 scripts/download_matrices.py
```

Docker (alternative). A pre-configured container is also available:

```
docker compose build
docker compose run artifact \
  ./scripts/run_all.sh
```

Experiment Workflow

A single command reproduces every experiment:

```
./scripts/run_all.sh
```

Outputs are placed in `output/ae_<timestamp>/`. Each figure or table can also be generated independently via a standalone script:

- `./scripts/run_figure_overall.sh` → Figure 8
- `./scripts/run_figure_nonsquare.sh` → Figure 9
- `./scripts/run_figure_mapping.sh` → Figure 10
- `./scripts/run_figure_breakdown.sh` → Figure 11
- `./scripts/run_figure_crossbar_width.sh` → Figure 12(a)
- `./scripts/run_figure_window_size.sh` → Figure 12(b)
- `./scripts/run_k_reordering.sh` → §IV-C ablation result

Evaluation and Expected Results

Because the simulator is fully deterministic, the CSV outputs should be bit-identical to the reference files shipped in `expected_results/`. The generated plots faithfully reproduce Figures 8–12 of the paper.

In addition, the `hardware/` directory ships SystemVerilog RTL for every SegFold module together with synthesis reports (area, power, and timing) produced by Synopsys Design Compiler targeting the ASAP 7nm standard cell library. These correspond directly to the hardware cost analysis in the paper.

Experiment Customization

Every experiment script supports `--jobs N` for parallelism control, `--config PATH` for alternative configurations, and `--timeout SEC` for per-run time limits. A single matrix can be evaluated directly:

```
./csegfold/build/csegfold \
  --config configs/segfold.yaml \
  --mtx-file benchmarks/data/suitesparse/
    ca-GrQc/ca-GrQc.mtx
```